

Fundamentals of Computer Programming

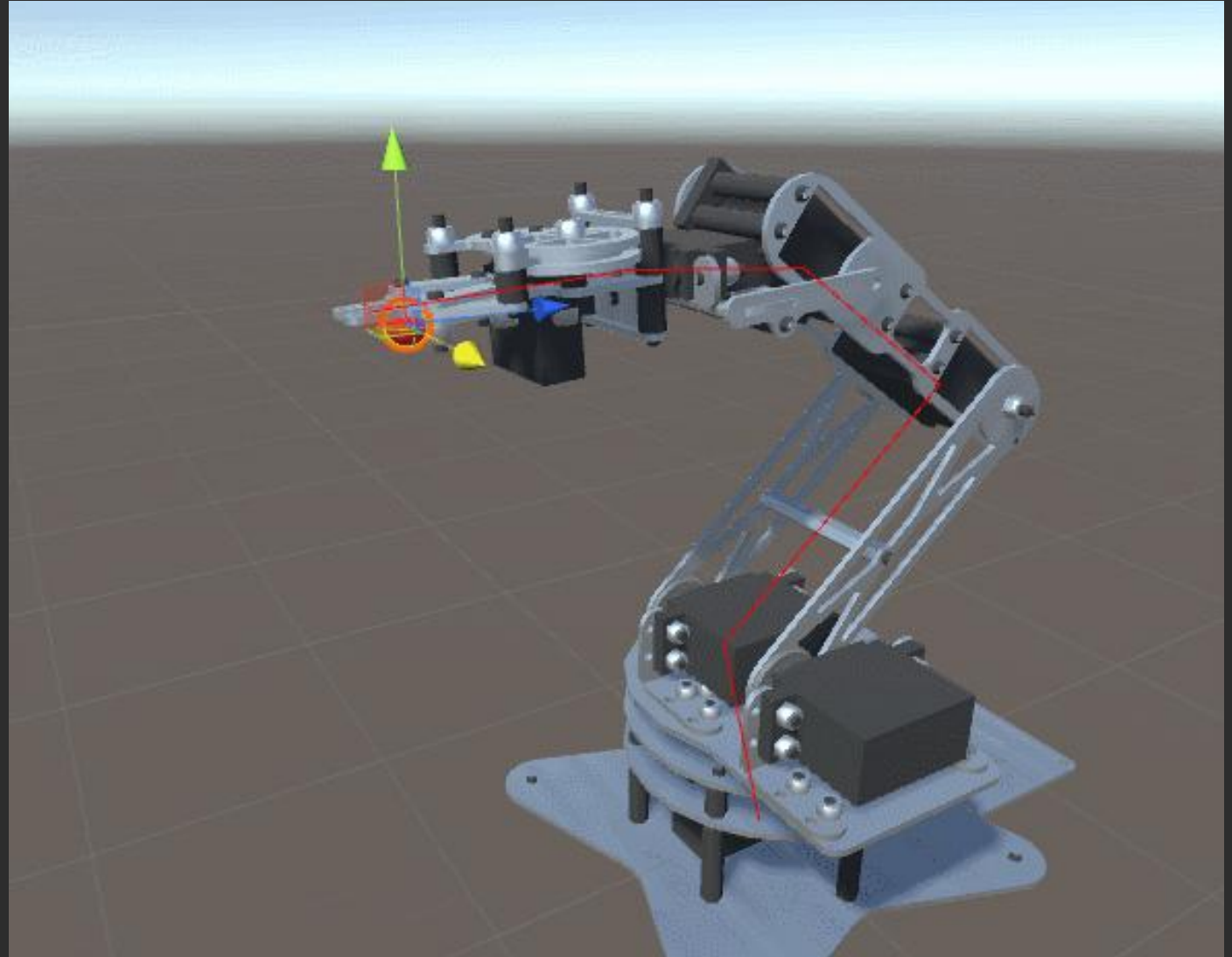
Design Problem: Forward Kinematics.

Week 2 | Lecture 3 (2.3)

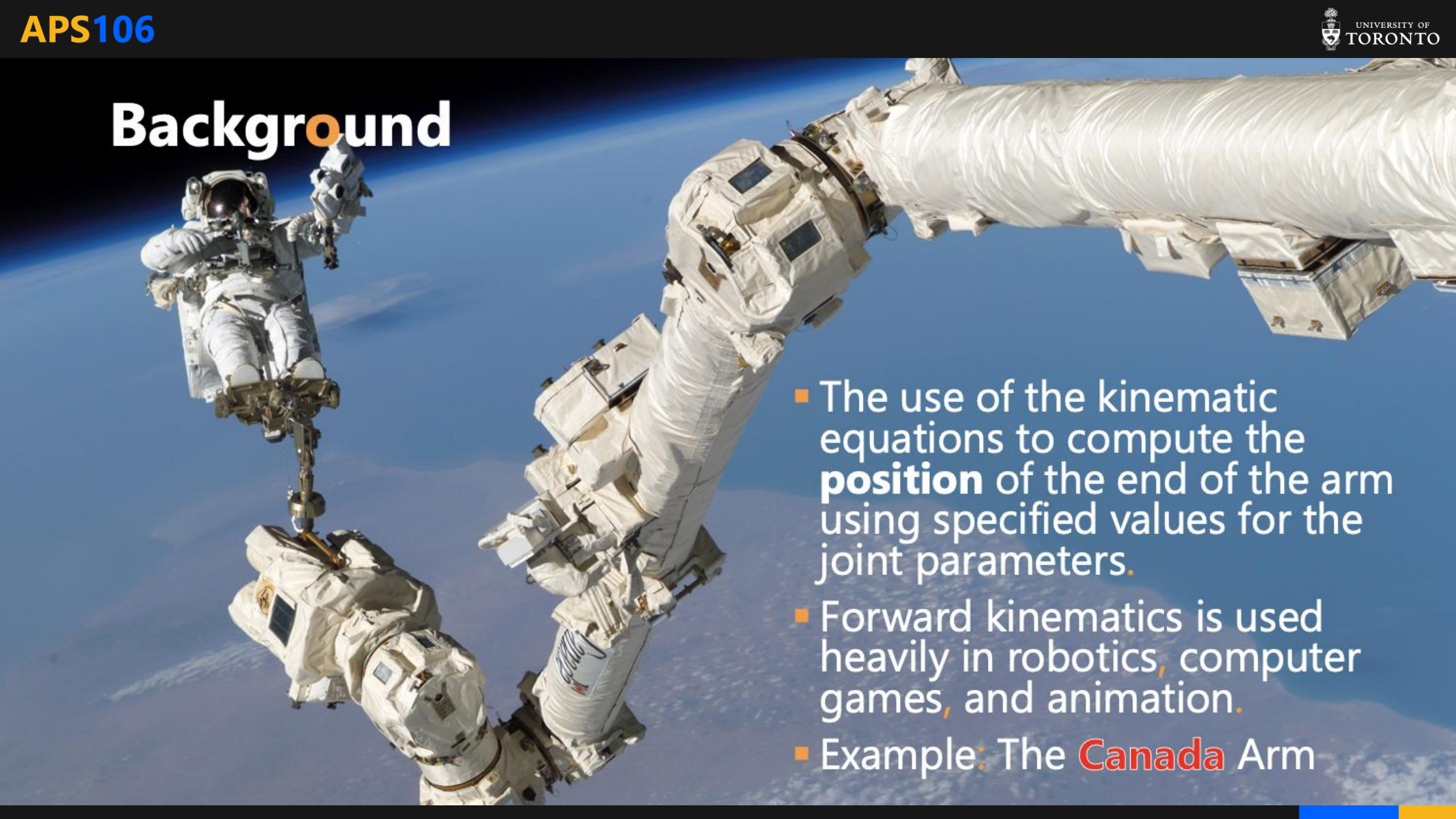
if nothing else, write **#cleancode**.

Agenda.

- Problem Background.
- Learning Objectives.
- Live Coding!

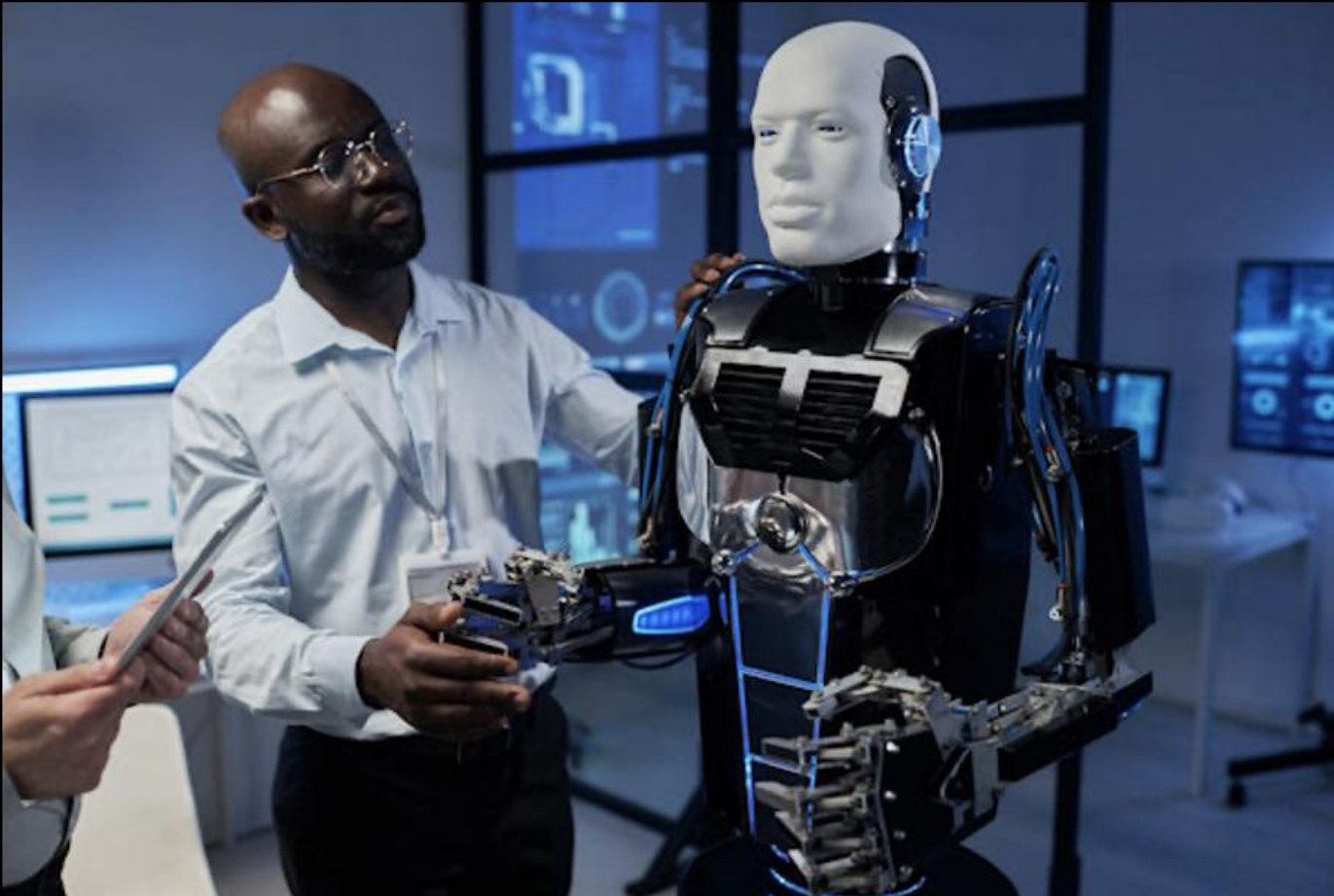


Background

- 
- The background image shows a full-body view of an astronaut in a white spacesuit working on the Canada Arm of the International Space Station. The arm is a long, white, cylindrical structure with various mechanical components and sensors. The astronaut is positioned on the left side of the frame, reaching towards the arm. The background is the Earth's surface, showing a blue sky and white clouds.
- The use of the kinematic equations to compute the **position** of the end of the arm using specified values for the joint parameters.
 - Forward kinematics is used heavily in robotics, computer games, and animation.
 - Example: The **Canada** Arm

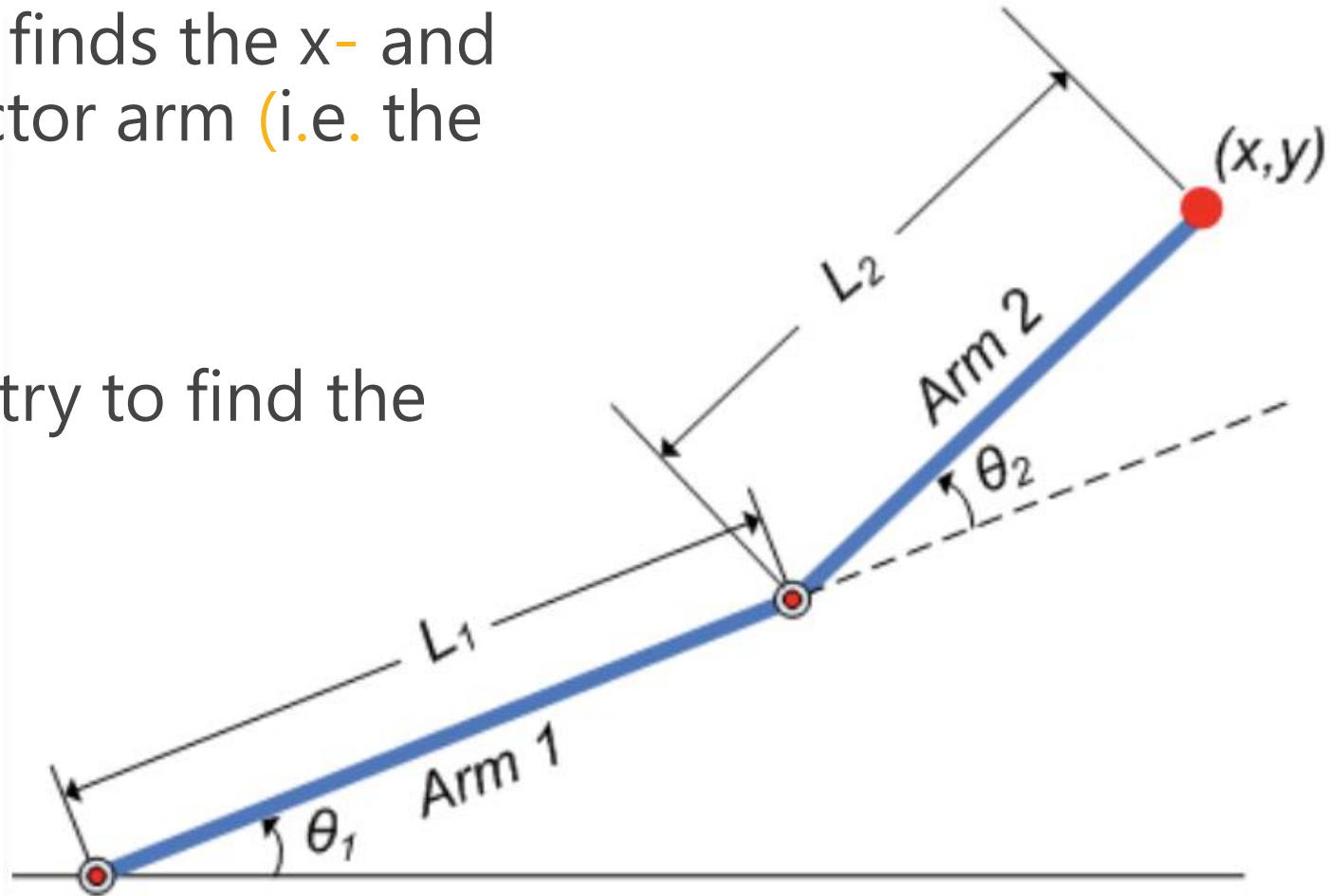
Other Cool Examples.

Humanoid and Industrial Robots, Animations and Gaming, 3D printing...

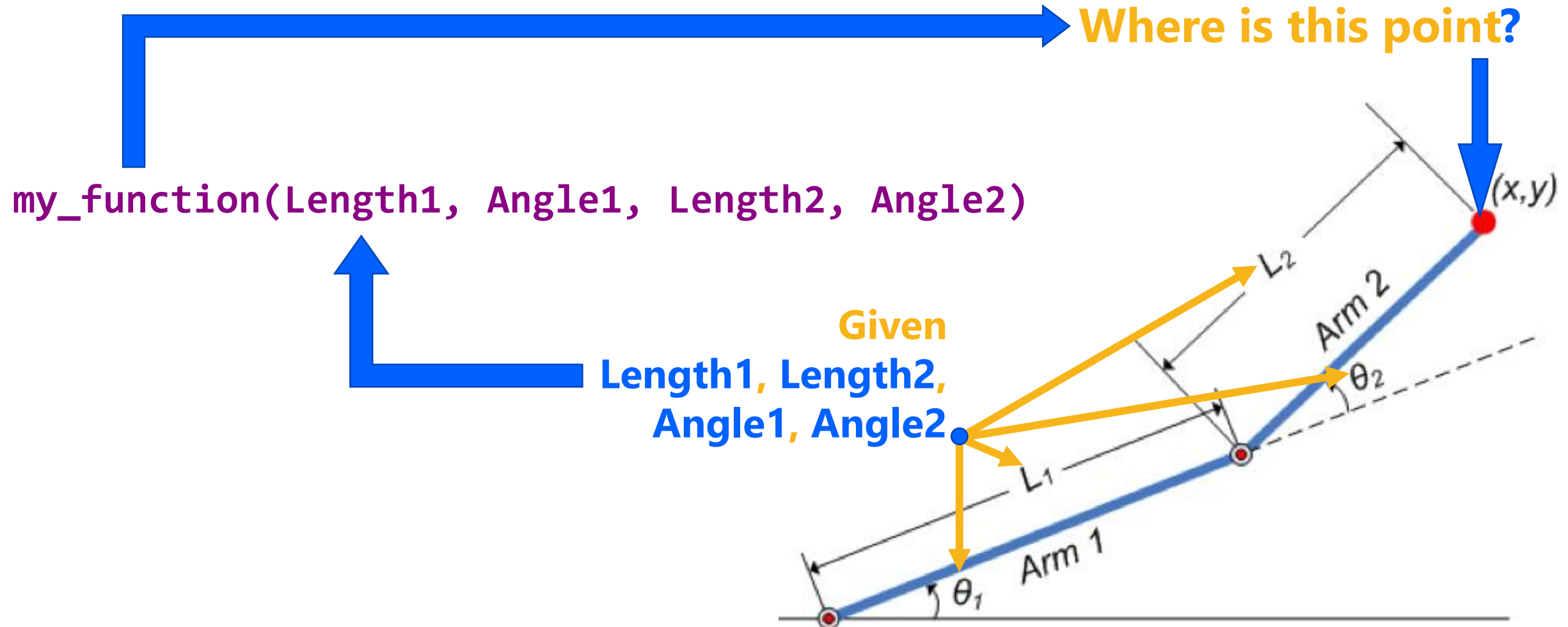


Requirements.

- Create a program that finds the x- and y- position of the effector arm (i.e. the end of Arm #2).
- Will rely on trigonometry to find the positions.



Requirements.

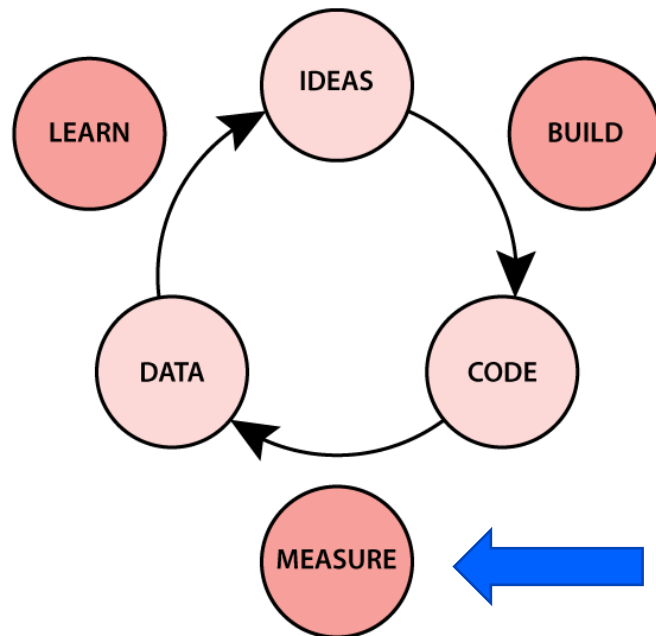


A Design Process for Programming.

- APS111/112, a key part of engineering, is the design of objects, processes, and systems.
- **Programming** is the design, implementation, testing, and documentation of a piece of software that solves a particular problem.
- The software might be part of a larger system (e.g., the avionics software of an aircraft, the accounting or human resources software of a business), but it represents the solution to a design problem (or part of a design problem).

A Design Process for Programming.

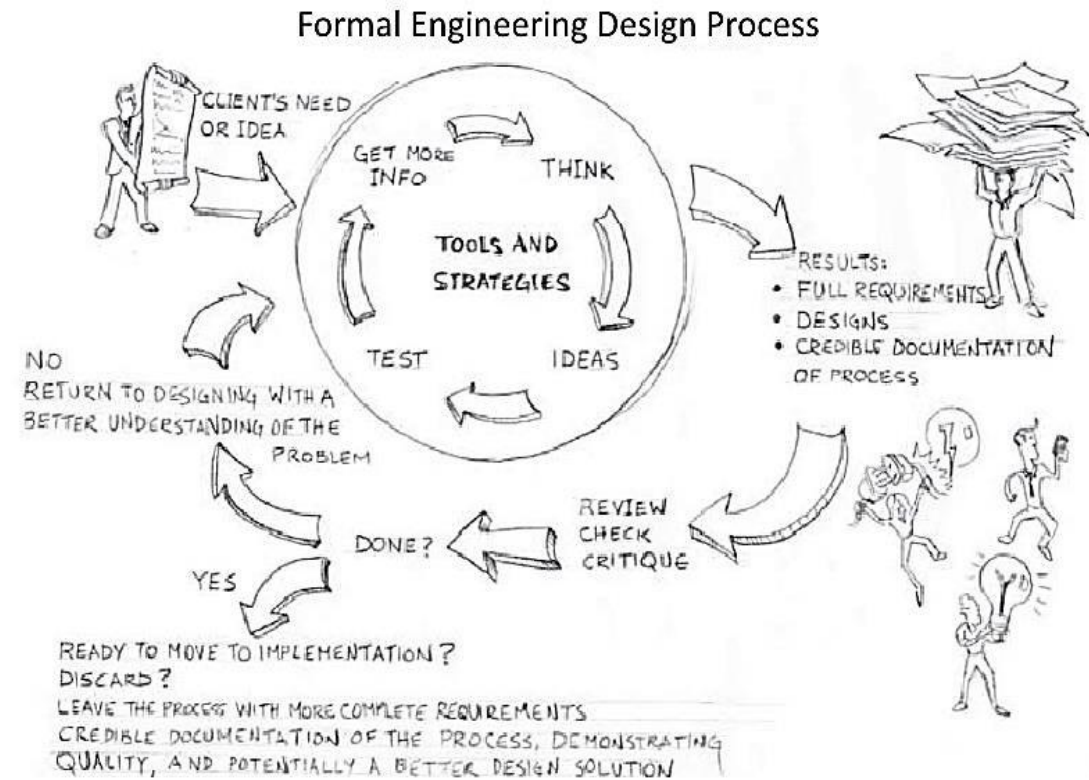
- We will approach programming as an engineering design process and adapt the process you have already seen in APS111/112.



Design is iterative.

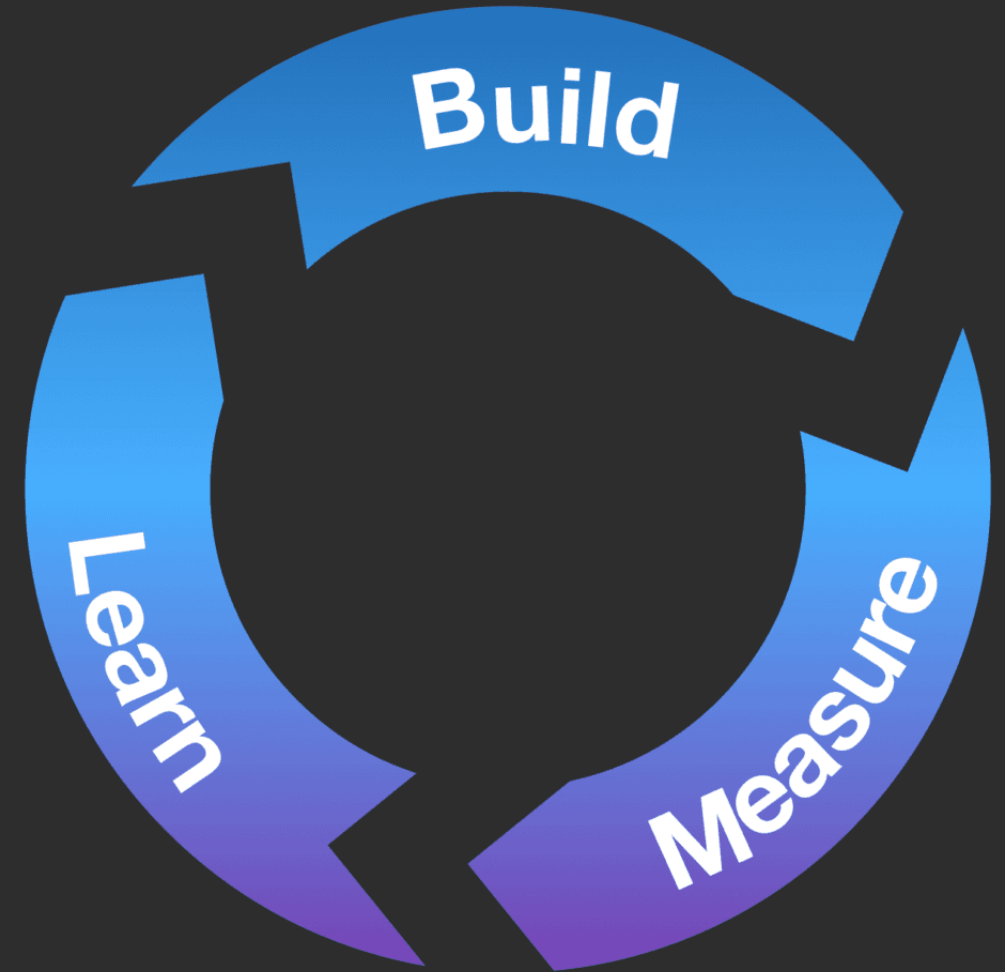
 Users.

Taken from: *Designing Engineers: An Introductory Text.*



A Design Process for Programming.

- The steps are as follows:
 - **Define the Problem.**
 - **Define Test Cases.**
 - **Generate Multiple Solutions.**
 - **Select a Solution.**
 - **Implement the Solution.**
 - **Perform Final Testing.**
 - **Iterate On Feedback.**



A Design Process for Programming.

- The steps are as follows:
 - **Define the Problem.**
 - Write down what the problem actually is.

**Open your
notebook**

Click Link:
**2. Define the
Problem**

A Design Process for Programming.

- The steps are as follows:
 - **Define Test Cases.**
 - Create some examples that reflect your code solving the problem: input and output.

**Open your
notebook**

Click Link:
3. Define Test Cases

A Design Process for Programming.

- The steps are as follows:
 - **Generate Multiple Solutions.**
 - At this point, a "solution" consists of an algorithm plan (the high-level sequence of steps defining what your algorithm will do) and a programming plan (the high-level sequence of steps that you will take to code the algorithm).

**Open your
notebook**

Click Link:
**4. Generate
Multiple Solutions**

A Design Process for Programming.

- The steps are as follows:
 - **Select a Solution.**
 - Based on the different algorithms and programming plans, decide which is the most promising.

**Open your
notebook**

Click Link:
5. Select a Solution

A Design Process for Programming.

- The steps are as follows:
 - **Implement the Solution.**
 - Start to execute your programming plan.
 - Test as you go!
 - You may realize that your algorithm plan doesn't solve the problem, or even that you do not understand the problem.
 - If so, go back to earlier steps.

**Open your
notebook**

Click Link:
**6. Implement the
Solution**

A Design Process for Programming.

- The steps are as follows:
 - **Perform Final Testing.**
 - Make sure that your original test cases, as well as any others that you have thought up, work.

**Open your
notebook**

Click Link:
**7. Perform Final
Testing**

A Design Process for Programming.

- The steps are as follows:
 - **Iterate On Feedback.**
 - As you or the others use your product, collect feedback.
 - Use feedback to improve the product.

**Open your
notebook**

**Click Link:
8. Iterate On
Feedback**

Fundamentals of Computer Programming

Design Problem: Forward Kinematics.

Week 2 | Lecture 3 (2.3)

if nothing else, write **#cleancode**.